

```

/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
#include <stdlib.h>

int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize,
               int* prerequisitesColSize, int* returnSize) {

    // Adjacency list
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    int* indegree = (int*)calloc(numCourses, sizeof(int));

    // Count outgoing edges
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graphSize[prereq]++;
        indegree[course]++;
    }

    // Allocate adjacency lists
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(graphSize[i] * sizeof(int));
        graphSize[i] = 0; // reset for filling
    }

    // Fill adjacency list
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][graphSize[prereq]++] = course;
    }

    // Queue for BFS
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;

    // Push nodes with indegree 0
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    // Result array
    int* order = (int*)malloc(numCourses * sizeof(int));
    int count = 0;

    // BFS
    while (front < rear) {
        int node = queue[front++];
        order[count++] = node;

        for (int i = 0; i < graphSize[node]; i++) {

```

```

        int next = graph[node][i];

        indegree[next]--;

        if (indegree[next] == 0) {
            queue[rear++] = next;
        }
    }

    // If cycle exists
    if (count != numCourses) {
        *returnSize = 0;
        free(order);
        order = (int*)malloc(0);
    } else {
        *returnSize = numCourses;
    }

    // Free memory
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }

    free(graph);
    free(graphSize);
    free(indegree);
    free(queue);

    return order;
}

```

Accepted Runtime: 0 ms	Accepted Runtime: 0 ms	Accepted Runtime: 0 ms
☒ Case 1 ☒ Case 2 ☒ Case 3	☒ Case 1 ☒ Case 2 ☒ Case 3	☒ Case 1 ☒ Case 2 ☒ Case 3
Input	Input	Input
numCourses = 4	numCourses = 2	numCourses = 1
prerequisites = [[1,0],[2,0],[3,1],[3,2]]	prerequisites = [[1,0]]	prerequisites = []
Output	Output	Output
[0,1,2,3]	[0,1]	[0]
Expected	Expected	Expected
[0,2,1,3]	[0,1]	[0]

Accepted 47 / 47 testcases passed

varshita24 submitted at May 27, 2026 15:16

Analysis

Solution

Runtime

11 ms | Beats 69.38%

Memory

14.25 MB | Beats 90.97%

